

LetterLab

Ethan Balda and Marcus Moore

Names and GitHubs

- Marcus Moore
 - GitHub: marcusm77
- Ethan Balda
 - GitHub: EthanBalda

Project Proposal

Concept: Create a puzzle game based around sorting a set of letters into alphabetical order. Each letter has a unique rule that must be used to sort the whole set.

Display: The app will have a starting screen that will display multiple buttons such as levels, help, a dictionary, and settings.

The dictionary displays the uncovered rules for each letter.

In a level, the player views the unsorted strings or grids of letters, the level's name, a retry and home button, and a move counter.

Upon completing a puzzle, a small menu appears displaying your moves, the moves to get par, and the moves to get a medal. The player can then continue to the next level, retry the current level, or return to the game's main menu.

The Letters (A - J)

- a; Apathetic

Does nothing. Other letters can interact with it.

- b; Basic : aba -> aab OR baa

Swaps places with a directly adjacent letter.

- c; Complex : aacaa -> caaaa OR aaaac

Swaps places with a letter one slot away from it.
One more space than b.

- d; Delete : adb -> ad_

Deletes whatever letter tries to interact with it.
Has some weird interactions

- e; Eat : abcec -> ab_cc

Eats an adjacent letter, deleting that letter and becoming it.

- f; Needs a word : abfcc -> bfcca OR cabfc

Moves the entire string or grid cardinally one space. If a letter would be knocked off the string, it wraps back to the start.

- g; Grab : abcagbb -> bcaagbb OR abacgbb OR acabgbb

Grabs an adjacent letter, pulls it adjacent to 'g', then moves all other letters back to fill space.

- h; Hold (This may need to be removed with grids) : ahb -> ah^b_

Holds a selected letter over its head, effectively removing it from the string or grid. Can swap out whatever is held with another letter. If it grabs a 'd', the 'h' and whatever it was holding get deleted, and the 'd' moves to the 'h' 's location.

- i; Immovable : acbi -> acbi

Cannot be interacted with by anything other than 'n'.

- j; Jump : abcjcba -> jbcacba OR abcacb

Like 'c', but with 1 more range.

The Letters (K - T)

- k; Kick : bkabba -> bkbbaa

The exact opposite of 'g'. Kicks an adjacent letter to the edge of the grid or string, then shifts every letter towards it to fill in the gap.

- l; Lovely (These should probably be ditched if we commit to grids.) : bacblcbb -> bbaclbbc

Pulls every letter into it whenever it is selected. Letters that would move to the selected 'l's slot instead go to the edges.

- m; Monstrous (These should probably be ditched if we commit to grids.) : bacbmccbb -> acbbmbccb

Opposite of 'l', they scare every letter one slot away. Any letters that would go off the grid, instead end up in the empty adjacent slots.

- n; No : aabcn -> abacn BUT NOT acabn

Adjacent letters to an 'n' effectively become 'a's. They have no special abilities anymore. This also applies to 'i's and makes this letter the only way to interact with 'i's. If two or more 'n's are next to each other, they 'no' each other effectively making both work as 'a's until they are no longer adjacent. There is no negative stacking, so 3 'n's cancel each other just like 2 'n's.

- o; Onomatopoeia : abaobba -> obaabba OR aboabba OR abaabbo

Can swap places with any selected vowels, except 'i'.

- p; Purge : aipo -> a___

Deletes self and cardinal neighbors when selected. Only deletes letters when it is selected first. It can be safely swapped with or interacted with by other letters.

- q; Queen

Can swap with any letter, barring 'i's and 'd's

- r; Remove : accdrca -> a__da_a

Removes every instance of a selected letter in a level, then transforms itself into an 'a'. Can only remove a letter that is adjacent to it. Can remove 'i's without issue.

- s; Spin : bsa -> asb

Swaps the letters adjacent to it with each other.

- t; Toggle : btc -> atc OR atb

Transforms the selected letter into the previous letter alphabetically. 'a's become 'z's. Can only 'toggle' letters adjacent to it.

The Letters (U - Z)

- u; Undo : btc -> ctc OR btd

The exact opposite of 't'. Changes an adjacent letter to its next letter alphabetically. 'z's become 'a's.

- v; Virus : abvac -> avvac OR abvvc

Exactly like a 'b', but whenever it swaps with a letter, that other letter becomes a 'v'. 'i's are of course immune to this. 'd's delete this letter then become a 'v'. 'h' that hold a 'v', delete that 'v' then become a 'v' themselves. t and u transform the letter, then become 'v's themselves.

- w; Waltz : aciwa -> awica -> waica

Always swaps left when selected. Jumps 'i's but does not swap with them.

- x; Xerox : caxb -> baxb

Selects two letters. Transforms the first letter into the second letter. Can transform letters into 'i's. Cannot transform 'i's. Same goes for 'n's. If it tries to transform a 'v', it instead makes both letters 'v's.

- y; Yeet : baaycib -> byaacib OR baacyib

Jumps the number of spaces equal to the number of vowels in the direction it wants to go.

- z; Zap

Can swap any two letters with each other.

Further Clarifications on Select Letters: D

d - e = both get deleted.

f = deletes the letter that would move into it.

g = d gets pulled to it like a normal letter, then the g is deleted.

h = deletes the currently held letter and the h, but moves to the h's spot.

k = gets thrown to the edge like other letters. Deletes the letter at the edge and not the 'k'

l = instead of going back to the edge, it deletes the 'l' if it goes all the way to 'l'.

m = just like 'l' except it gets stuck at the edge and deletes incoming letters.

d - n = no longer deletes anything.

p = deletes 'd' without issue.

r = removes all 'd's from the level, but is deleted in the process rather than becoming an 'a'.

s = deletes the letter it would swap with, not the 's'.

t/u = nothing weird.

v = deletes the 'v' but becomes a 'v'.

x = deletes whatever the other letter is. Still transforms into the other letter if it was selected first.

z = Moves to the other letter's location but deletes the other letter.

Further Clarifications on Select Letters: H

h - d= deletes the 'h' and whatever letter it was holding. The 'd' moves to the 'h's position.

h= if this 'h' you are trying to hold also has a letter, that letter stays held by the 'h' held by the 'h'. The consequences for this will be something we regret later.

n= if picked up, the 'h' can no longer pick up any other letters.

r = if the selected letter is held by 'h', the letter is removed allowing 'h' to hold a new letter.

v= deletes this letter, but the 'h' and the held letter both become 'v's.

x= holding this does nothing special. However, if an 'h' holding a letter is xerox'd, the held letter is also copied over to the new 'h'.

Further Clarifications on Select Letters: V

v - a/b/c/j/q/s/z= swap positions
like normal, then the
other letter becomes a
'v'.

d= deletes the 'v' then
becomes a 'v'.

e= becomes a 'v' without
deleting the original 'v'.

g= it gets pulled to the 'g' like a
normal letter, then
transforms the 'g' into a 'v'.

h= swaps places with the
held letter like normal.
The held letter then
becomes a 'v'. The 'h'
then also becomes a 'v',
deleting the held 'v'.

k= kicks the 'v' to the edge. The letter at
the edge becomes a 'v' then every letter
moves to fill the gap.

l = Instead of going to the edge when
reaching the 'l', it transforms the 'l', then
all letters adjacent to the 'l'. Note that
this means the rest of the string or grid
do not move closer to the 'l' when a 'v'
would move into the 'l'.

m=when it wraps around
an edge back to 'm',
the 'm' becomes a 'v'
and all letters
adjacent to the 'm'
also become a 'v'.

n= if next to the 'v', the
'v' cannot infect any
letters. If the 'v' is
swapped away from
the 'n', it still cannot
infect anything until
its next turn. If a letter
is adjacent to 'n' at
the end of its swap with
a 'v'. It does not get
infected.

w= if a 'w' swaps with it, the 'w'
does become a 'v'.

x = both letters become 'v's
regardless of what the other letter
is.

z = treat the swap as any other 'v' swap.

Gameplay

The player selects a selectable letter, then another letter to interact with. These interactions count as moves, which are counted in the corner. The goal of the player is to turn a garbled set of letters into alphabetical order in as few moves as possible.

Completing a level under a specified par unlocks the next. Doing particularly well on a puzzle also rewards a gold medal.

Main Menu

This activity is the main menu where the user is brought when the app first loads. The user is given options to go to the dictionary(a help screen), to play the game or quit the app.

Dictionary

If the dictionary button is selected the user is brought to a screen showing a list of the rules for each of the letters. This is simply done by pulling text from a txt file and displaying it on the screen. This screen also contains a back arrow that leads to the main menu

LevelSelectActivity

When the play button is clicked the user is first brought to a level select screen where all our available levels are listed. Although you can only play levels if you have beaten the level before it. We have done this by giving each level a value of 0,1, or 2. 0 meaning the level is locked, 1 the level is unlocked but not beaten, and 2 if the level is beaten. The method displayLevelButtons uses this by initializing all levels to 0 besides level 1 and when a level is beaten the value then goes to 2 and if a levels value is 2 it increases the level after its value to 1.

GameActivity

This is where all the game logic is stored. By using the Shared Preferences variable a built in kotlin variable it automatically saves data such as the values for the levels on each level. So the users data automatically saves after they complete a level. We also use our api and network connection here where our chosen api UnsplashApi pulls images from the internet and makes it a random background for our levels.

GameActivity Level Setup

To create the game, I had Ethan create an LxW grid of buttons. From here, I then went in and had that grid read from an array a string. This string would function as the instructions to build a level, by literally naming each button after a letter in that string. I then also assigned each button a tag based on their position in the string. While the button layout is a 2-Dimensional array, the strings are all one dimensional. As such, a lot of the code to build the levels is just converting a one-dimensional array into a 2d one based on the returns in the level string.

GameActivity Simple Letter Mechanics

I assign logic to each letter button based on the letter it holds. For 'b', 'c', and 'j' this was exceptionally easy thanks to all 4 of them being very mechanically similar. When one of them is clicked, the grid disables all buttons not within swapping range of these letters. When the player clicks one of the only remaining active buttons, it simply swaps the two buttons' letters. It then resets the grid by disabling every button, so only interactable buttons are re-enabled.

For every other interactable letter, they use 'b' as a template. For example, the letter 'e' effectively works identically to 'b', except it replaces its swapee with a '_' character instead of itself.

'h' similarly can interact with any piece on the board, holding a letter in place, then swapping its held letter with any other letter on the board.

GameActivity Complex Letter Mechanics

The remaining letters implemented: 'f', 'g', 'k', 'l', and 'm' were all far more complicated. In fact, 'f' got its own entire function.

Starting with 'f', rolling everything on the board in a given direction became very complicated very quickly due to each letter being tagged with their 1-dimensional coordinates, forcing a 1.5-dimensional approach to shifting. Thankfully, once this was solved, the other four letters could be implemented using a single modified version of 'f's function. Where 'f' moves the whole grid, these other letters move single rows or columns, or even less. The full code is still very useful especially for looping letters to the front or back of their row or column.

GameActivity Deleting Letters

The letter 'd' can be thought of less as a letter compared to the others, and more as a modifier. It required unique modifications to every single prior letters' code. Namely, the swaps need to become '_'. This was as simple as a single if check.

With 'f', the letter shifting into 'd' needs to be deleted while 'd' itself does not move, creating the '_' after it. This then requires several checks to verify the direction of the shift and position of the new '_'.

For 'l' and 'm' much of the same was used. However, for 'k' and 'g', a very notable exception exists to make for a fun interaction. Unlike all other letters, 'k' and 'g' do get to interact directly with 'd'. If 'd' is kicked, it deletes every letter it passes. If it is grabbed, it deletes every letter in front of it. This required a bit more effort in regards to tweaking the loops for those two letters, as now effectively a new whole loop needs to be interpreted.

Finally, 'h' becomes even more powerful. When holding a 'd' it loses the ability to swap and instead just deletes any letter that tries to swap with its hold.

GameActivity Winning

Finally, to check that a player has won, each row and column is checked to be in either alphabetical or reverse-alphabetical order. Due to the presence of 'd' and by extension '_', '_' are treated as nothing and therefore can be anywhere in the grid.

Ethan then used this function's output to determine when to save data and boot the player back to level select.